



Cypher Launchpad

Security Review



Disclaimer

Security Review

Cypher Launchpad



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

Cypher Launchpad

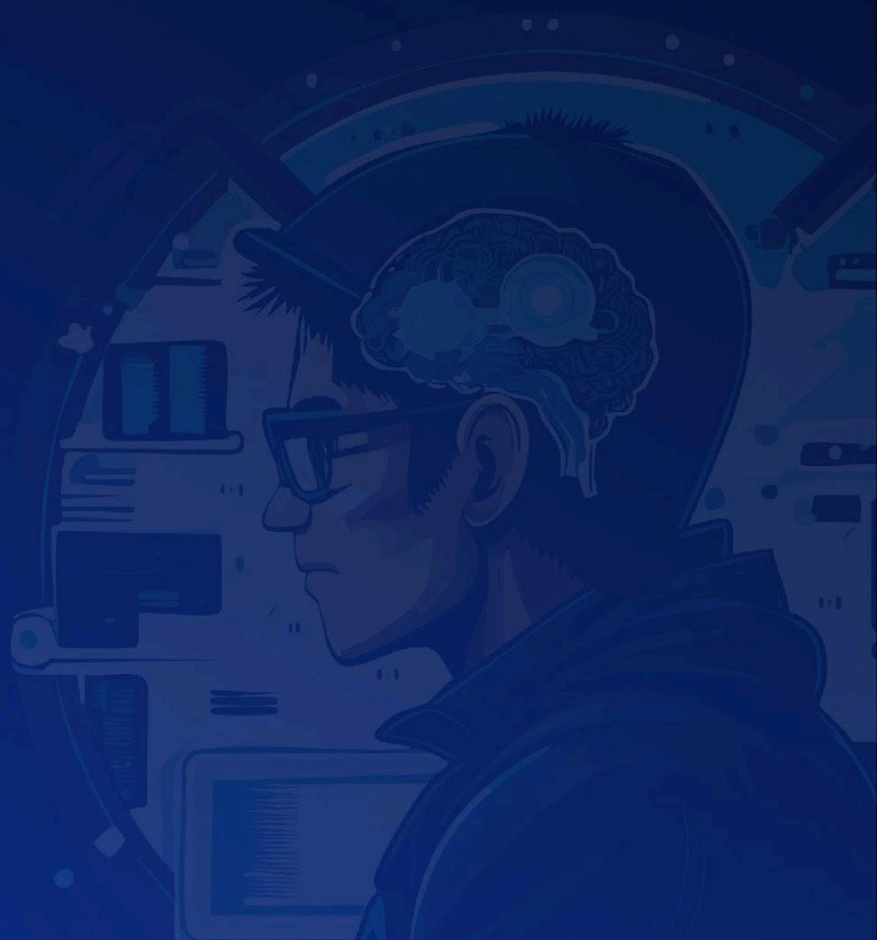


Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S--M01	18
3S--M02	20
3S--L01	22
3S--N01	24
3S--N02	26
3S--N03	28

Summary Security Review

Cypher Launchpad



Summary

Three Sigma audited Cypher in a 0.4 person week engagement. The audit was conducted on 08/04/2026.

Protocol Description

Cypher Factory is an Ethereum Mainnet token factory built on top of the Cypher DEX. Whenever a new token is launched, a new Algebra V4 liquidity pool is created, setting its ``sqrtPriceX96`` and it enters a `_Bonding Curve_` phase where its params are inherited from the `BCTokenFactory.sol` config. During the `_Bonding Curve_` phase, users are able to trade within a virtual bonding curve until the token ``MAX_SUPPLY`` is reached (which is equivalent to reaching a target market cap or ``TARGET_ETH``).

Once the ``MAX_SUPPLY`` is reached, all the ETH available plus an initial BCToken supply are deployed as liquidity to the created LP and the positions are transferred to a Harvester contract which locks them but enables a privileged account or contract to harvest the fees. BCToken supply is deployed from the `_seed tick_` to MAX or MIN tick and ETH is deployed from `_seed tick_` to the `_initial expected tick_` which matches the first price of the Bonding Curve.

Scope Security Review

Cypher Launchpad

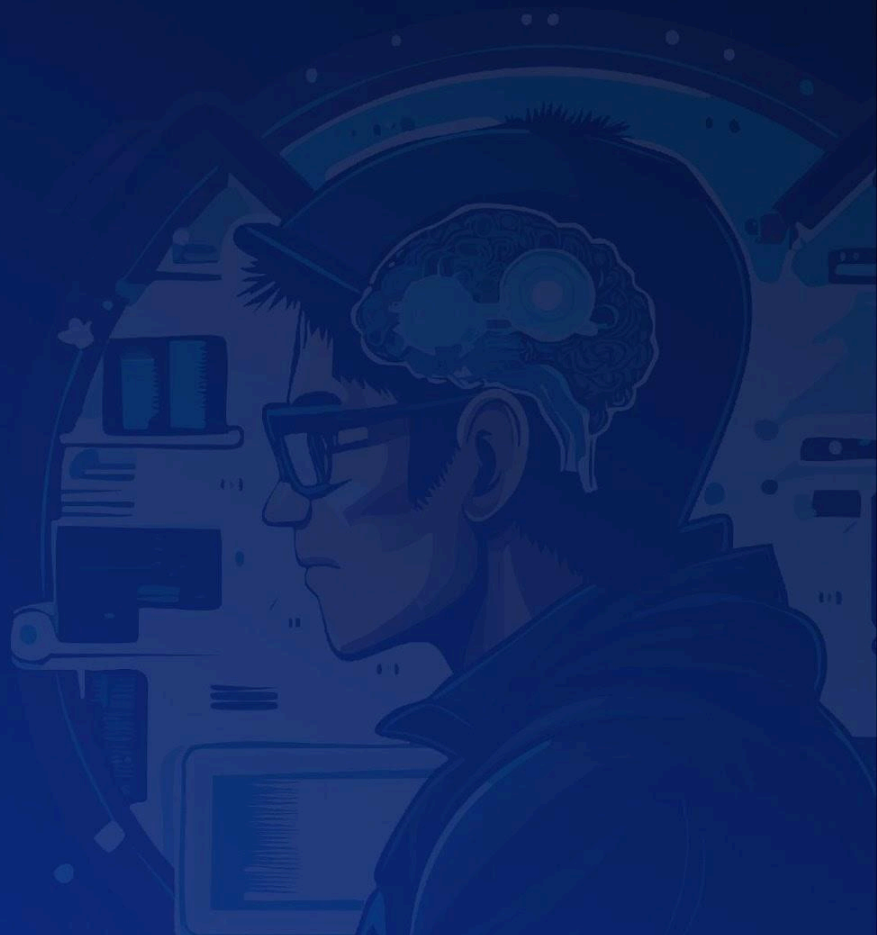


Scope

Filepath	nSLOC
src/algebra/plugin/AlgebraFeePluginV1.sol	99
src/algebra/plugin/AlgebraFeePluginFactoryV1.sol	169
src/BCToken.sol	151
src/LiquidityManager.sol	152
src/BCTokenFactory.sol	181
src/BCTokenDeployer.sol	25
Total	777

Methodology Security Review

Cypher Launchpad



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard Security Review

Cypher Launchpad



Project Dashboard

Application Summary

Name	Cypher
Repository	https://github.com/CypherIndustries/factory-contracts
Commit	b8262b6
Language	Solidity
Platform	Ethereum Mainnet

Engagement Summary

Timeline	08/04/2026
Nº of Auditors	2
Review Time	0.4 person weeks

Vulnerability Summary

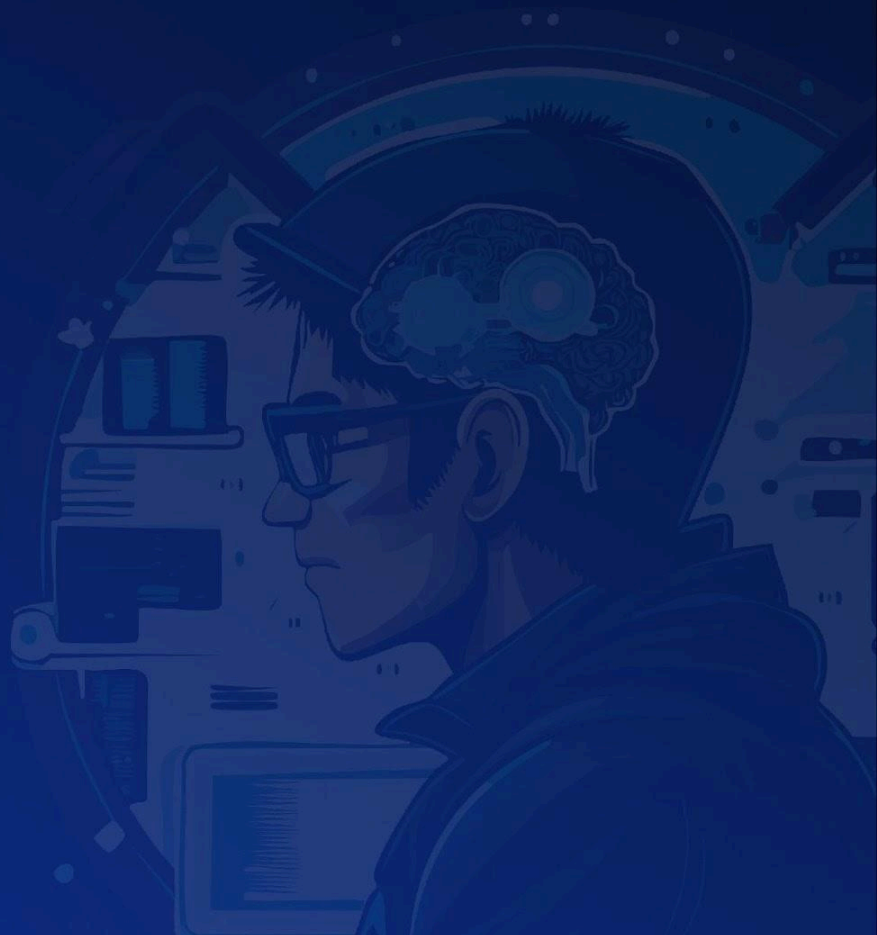
Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	2	2	0
Low	1	1	0
None	3	2	1

Category Breakdown

Suggestion	3
Documentation	0
Bug	3
Optimization	0
Good Code Practices	0

Risk Section Security Review

Cypher Launchpad



Risk Section

No risks were identified.

Findings

Security Review

Cypher Launchpad



Findings

3S-M01

Hardcoded **creator** recipient in **_seedLP** graduation fee transfer leads to **alternativeFeeRecipient** being bypassed

Id	3S--M01
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #41da92c .

Description

BCToken::_seedLP runs once when the bonding curve fills, wrapping the accumulated ETH into WETH, distributing the graduation fees to the creator, the protocol and the caller, and then forwarding the remaining balances to the **LiquidityManager** for pool seeding. The creator-side share is intended to flow to whichever address the token has configured as its fee destination.

The contract already centralises that resolution in **BCToken::getFeeRecipient**, which returns **alternativeFeeRecipient** when it is set and falls back to **creator** otherwise. Other protocol components rely on this helper: **AlgebraFeePluginV1** calls **IBCTokenFeeRecipient.getFeeRecipient()** when routing post-graduation swap fees, so any configured alternative recipient is honoured from that point onward.

_seedLP does not follow the same convention, it transfers the graduation creator fee directly to the **creator**.

As a result, when a token has a non-zero **alternativeFeeRecipient**, the one-time graduation fee is still paid to **creator**, diverging from the recipient used by every subsequent swap-fee payout. Ongoing fees reach **alternativeFeeRecipient** through the Algebra plugin while the initial lump sum does not, leaving the configured recipient short by **creatorFee / 10_000** of the bonding-curve balance at graduation. The misrouting is silent (no revert, no event discrepancy) and cannot be corrected after the fact because **_seedLP** only executes once per token.

Recommendation

Resolve the graduation fee recipient through the same helper used everywhere else in the protocol. Because **getFeeRecipient** is currently declared **external**, it should also be widened to **public** so **_seedLP** can call it directly without an external self-call.

```
- function getFeeRecipient() external view returns (address) {
+ function getFeeRecipient() public view returns (address) {
    address alt = alternativeFeeRecipient;
    return alt != address(0) ? alt : creator;
}

- IERC20(weth).safeTransfer(creator, ethBalance * creatorFee / 10_000);
+ IERC20(weth).safeTransfer(getFeeRecipient(), ethBalance * creatorFee / 10_000);
```

3S-M02

Using the cliff timestamp as the vesting time reference leads to under-vested releases and delayed schedule completion

Id	3S--M02
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #88e92e4 .

Description

TokenVesting implements a linear vesting curve with an initial cliff period. In **TokenVesting::createVestingSchedule**, the caller supplies **start** (an absolute timestamp), **cliff** (a relative offset from **start**), and **duration** (the total length of the vesting period in seconds). This is confirmed by the natspec in **ITokenVesting (@param duration The total duration of the vesting period in seconds.)** and by the input validation **if (duration < cliff) revert CliffExceedsDuration(cliff, duration);**, which only makes sense if **cliff** is a sub-interval of **duration**. The function stores the absolute cliff timestamp as **cliffTimestamp = start + cliff**, so the expected end of the schedule is **start + duration**, and at any point in time the vested fraction should be **(block.timestamp - start) / duration**.

TokenVesting::_computeReleasableAmount instead uses the stored **cliff** field (the absolute cliff timestamp) as the origin for its linear interpolation. The final-vesting check **block.timestamp >= vestingSchedule.cliff + vestingSchedule.duration** shifts the effective schedule end from **start + duration** to **start + cliffOffset + duration**, and the interpolation variable **timeFromStart = block.timestamp - vestingSchedule.cliff** measures elapsed time from the cliff rather than from **start**. As a result, the vested amount computed at any time **t** is **amountTotal * (t - cliff) / duration** instead of **amountTotal * (t - start) / duration**, so the portion corresponding to the cliff interval (**amountTotal * cliffOffset / duration**) is never credited linearly and the schedule is not fully vested until **cliffOffset** seconds after the intended end.

The concrete effects are: beneficiaries lose the cliff chunk that should become releasable the moment the cliff is reached, every release before the end of the schedule returns less than intended, and full vesting is postponed by the length of the cliff. For a schedule with **duration = 365 days** and **cliff = 90 days**, beneficiaries receive **0** at the cliff (instead of roughly 24.6% of **amountTotal**) and only receive the full balance at **start + 455 days** rather than **start + 365 days**.

Recommendation

Use **vestingSchedule.start** as the time origin in

TokenVesting::_computeReleasableAmount, both when detecting the end of the schedule and when computing the elapsed time used for the linear interpolation. The early-return guard on **block.timestamp < vestingSchedule.cliff** already ensures nothing is released before the cliff, so switching the reference to **start** preserves the cliff semantics while restoring the intended linear curve.

```

// After vesting period: all remaining tokens are releasable.
-   if (block.timestamp >= vestingSchedule.cliff + vestingSchedule.duration) {
+   if (block.timestamp >= vestingSchedule.start + vestingSchedule.duration) {
        return vestingSchedule.amountTotal - vestingSchedule.released;
    }
// During vesting: calculate based on elapsed slice periods.
-   uint256 timeFromStart = block.timestamp - vestingSchedule.cliff;
+   uint256 timeFromStart = block.timestamp - vestingSchedule.start;

```

3S-L01

Missing per-token snapshot of **defaultPoolEnabled** leads to liquidity seeding revert on admin toggle

Id	3S--L01
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Addressed in #834d8ee .

Description

LiquidityManager governs the creation and seeding of Algebra pools for every BCToken. The lifecycle is split between two entry points called at different moments:

LiquidityManager::createAndInitializePools, invoked at token deployment, and **LiquidityManager::seedLP** (which delegates to **LiquidityManager::_seedPools**), invoked later when the bonding curve graduates. Both code paths read the protocol-wide **defaultPoolEnabled** flag from **BCTokenFactory** live, without caching the value observed at deployment.

At deployment time, **createAndInitializePools** branches on the current flag and, if **true**, calls **IAlgebraFactory.createPool(token0, token1, "")** to provision an additional default (POL) pool alongside the custom pool. At graduation time, **_seedPools** branches on the same live flag to decide whether to mint 95% of the liquidity into the custom pool and 5% into the default pool, or to send 100% to the custom pool.

Because the owner can call **BCTokenFactory::setDefaultPoolEnabled** at any point, the two reads can observe different values for the same token. In the most damaging direction, the flag is **false** when a token is deployed (so no default pool is created) and then flipped to **true** before graduation. When **_seedPools** executes for that token it enters the POL branch and forwards liquidity to a default pool address that was never initialized, so the subsequent **IPositionManager.mint** call reverts. Every graduation attempt for the affected token keeps reverting until the admin restores the flag, during which the token cannot graduate. The opposite toggle (**true** → **false** after deployment) bypasses the POL branch even though a default pool was provisioned, leaving the pool empty of protocol-owned liquidity.

The bug is an operational misconfiguration rather than a direct value-loss vector: the owner can unblock graduation by toggling the flag back to its original state, and no funds are lost.

Recommendation

The branch taken at seeding time must match the branch taken at deployment time for each individual token, independent of later admin changes to the global flag. Introduce a per-token snapshot in **LiquidityManager** such as **mapping(address => bool) public hasDefaultPool**, write it in **LiquidityManager::createAndInitializePools** based on the flag value observed at that moment, and read from it in **LiquidityManager::_seedPools** instead of querying **IBCTokenFactory(TOKEN_FACTORY).defaultPoolEnabled()** again.

3S-N01

Missing **bcTokens** authorization check in **LiquidityManager::seedLP**

Id	3S--N01
Classification	None
Category	Suggestion
Status	Addressed in #2f27cc0 .

Description

LiquidityManager::seedLP is the entry point invoked by a **BCToken** once its bonding curve graduates, pulling the accumulated token and **WETH** balances from the caller and minting the two one-sided Algebra v3 positions that seed both the custom pool and the shared default (POL) pool. Its sibling, **LiquidityManager::createAndInitializePools**, correctly gates access by checking **if (!IBCTokenFactory(TOKEN_FACTORY).bcTokens(msg.sender)) revert Unauthorized();** at the top of the function, ensuring only canonical **BCToken** instances registered by the factory can drive pool creation.

seedLP omits this guard entirely. It treats **msg.sender** as the token address without verifying that the caller is registered in the **bcTokens** mapping, then proceeds to pull balances from the caller (which the caller controls via **approve**) and call **_seedPools**, which eventually reaches **IPositionManager.mint(...)**. Any attacker contract implementing the **IBCToken** view surface (**harvester()**, **lp()**, **VIRTUAL_BALANCE()**, **MAX_SUPPLY()**, **PROTOCOL_FEE()**, **CREATOR_FEE()**) can call **seedLP** directly, supplying its own ERC-20 and **WETH**, and reach the minting path.

No direct fund extraction is possible because the attacker spends only tokens it owns, and the positions are minted to the attacker-controlled **harvester()** address. The residual damage is confined to protocol-invariant violations: the attacker can push arbitrary tick-state into the shared default pool (reached via the **address(0)** deployer branch in **_seedPools** when **defaultPoolEnabled()** is true), mint positions whose **recipient** is a garbage **harvester()**, and generally break the assumption that only graduated **BCTokens** ever touch **_seedPools**.

Recommendation

Mirror the guard already present on **LiquidityManager::createAndInitializePools** by checking **bcTokens(msg.sender)** at the top of **seedLP** and reverting with **Unauthorized** when the caller is not a registered **BCToken**.

```
function seedLP(uint256 bcTokenBalance, uint256 wethBalance) external {  
+   if (!IBCTokenFactory(TOKEN_FACTORY).bcTokens(msg.sender))  
+       revert Unauthorized();  
+  
    address token = msg.sender;  
    IBCToken coin = IBCToken(token);
```

3S-N02

Reading **address(this).balance** in **_seedLP** instead of tracked reserves

Id	3S--N02
Classification	None
Category	Suggestion
Status	Addressed in #3a97035 .

Description

BCToken::_seedLP is the one-shot graduation routine that runs once the bonding curve has fully sold out. It wraps the accumulated ETH into WETH, pays the creator, protocol and caller shares of the graduation fee, then forwards the remaining WETH and the contract's **BCToken** balance to the **LiquidityManager** to seed the post-graduation pool. Every ETH amount that flows out of this function, the fee payouts and the seed deposit alike, is derived from a single local variable at line 270:

```
uint256 ethBalance = address(this).balance;
```

The rest of the contract does not trust the raw balance. Buys and sells on the curve are accounted through the private **accETH** variable, which is incremented in **BCToken::mint** and decremented in **BCToken::burnCurve**, and the protocol already exposes **BCToken::TVL** as **accETH - VIRTUAL_BALANCE** to return the real, curve-owned ETH excluding the virtual reserves used only for pricing. **_seedLP** is the only place where the contract takes the balance at face value, so it is also the only place where the two accounting views (raw balance vs. **accETH**) can drift apart.

The drift can be induced by forced ETH transfers. **BCToken** has no **receive** or **fallback** function, so regular **call{value:}** sends cannot reach it, but **selfdestruct** from a funded helper and coinbase/withdrawal credits can still increase **address(this).balance** without touching **accETH**. Any wei pushed into the contract before the graduating buy lands will be included in **ethBalance**. No direct theft path is apparent in the current layout, since the donated ETH ends up split between the fee recipients and the pool rather than being redirected to an attacker, but the contract is silently operating on an untrusted number.

Recommendation

Use the contract's own accounting to derive the amount to wrap and distribute, so **_seedLP** is consistent with every other curve-facing function and is not affected by force-sent ETH.

BCToken::TVL already returns the correct figure (**accETH - VIRTUAL_BALANCE**), so the local variable can be replaced directly:

```
-    uint256 ethBalance = address(this).balance;  
-    IWETH(weth).deposit{value: ethBalance}();  
+    uint256 ethBalance = TVL();  
+    IWETH(weth).deposit{value: ethBalance}();
```

3S-N03

Creator buy cap is derived from total max supply instead of bonding curve supply

Id	3S--N03
Classification	None
Category	Suggestion
Status	Acknowledged

Description

The **BCTokenFactory::_creatorBuy** function allows the token creator to purchase tokens from the bonding curve immediately after deployment. The amount of tokens the creator is allowed to buy is capped by **maxTokens**, which is computed as a percentage (**maxCreatorBuyBps**) of **_maxSupply**.

However, not all of **_maxSupply** is available on the bonding curve. A portion (**_initialSupply**) is reserved to seed liquidity and is never sold through the curve. The actual number of tokens available for purchase on the bonding curve is **_maxSupply - _initialSupply**. Because **maxTokens** is derived from the total **_maxSupply** rather than the bonding curve supply, the effective cap on the creator buy is larger than intended relative to the tokens actually purchasable, and in certain configurations the cap becomes entirely non-restrictive.

Steps to Reproduce

1. Configure **maxCreatorBuyBps** to **2500** (25%).
2. Set **_maxSupply** to 100,000,000 tokens and **_initialSupply** to 80,000,000 tokens. The bonding curve holds 20,000,000 tokens.
3. Call **BCTokenFactory::deploy** with enough ETH to purchase a large amount of tokens.
4. **_creatorBuy** computes **maxTokens = 100,000,000 * 2500 / 10,000 = 25,000,000**.
5. The cap (25,000,000) exceeds the entire bonding curve supply (20,000,000), rendering the creator buy restriction ineffective. The creator can buy the full bonding curve supply and push the token to graduation immediately at launch.

Recommendation

Derive **maxTokens** from the bonding curve supply (**_maxSupply - _initialSupply**) instead of **_maxSupply**. This ensures the cap is proportional to the tokens actually available for purchase, regardless of how **_maxSupply** and **_initialSupply** are configured.

```
function _creatorBuy(address token, address tokenCreator, uint256 ethAmount)
internal {
-   uint256 maxTokens = _maxSupply * maxCreatorBuyBps / 10_000;
+   uint256 maxTokens = (_maxSupply - _initialSupply) * maxCreatorBuyBps /
10_000;
    IBCToken(token).setTransferWhitelist(address(this), true);
    uint256 tokensBought = bondingCurve.buy{value: ethAmount}(token, address(0),
0);
    if (tokensBought > maxTokens) revert CreatorBuyExceedsCap();
    IERC20(token).safeTransfer(tokenCreator, tokensBought);
    IBCToken(token).setTransferWhitelist(address(this), false);
    emit CreatorBuy(tokenCreator, token, ethAmount, tokensBought);
}
```